

Study Abroad Discovery Platform

Complete Product, UI/UX, Technical Architecture, Backend Schema, ER Diagram and Implementation Specification

Working product name: AbroadHUB

Document version: 1.0 | Date: 24 August 2026

Purpose

This document consolidates the full project specification into one implementation reference. It covers product goals, user flows, frontend design, backend architecture, PostgreSQL schema, security, authentication, API contracts, ER relationships, responsive behavior, light/dark mode, and the implementation sequence.

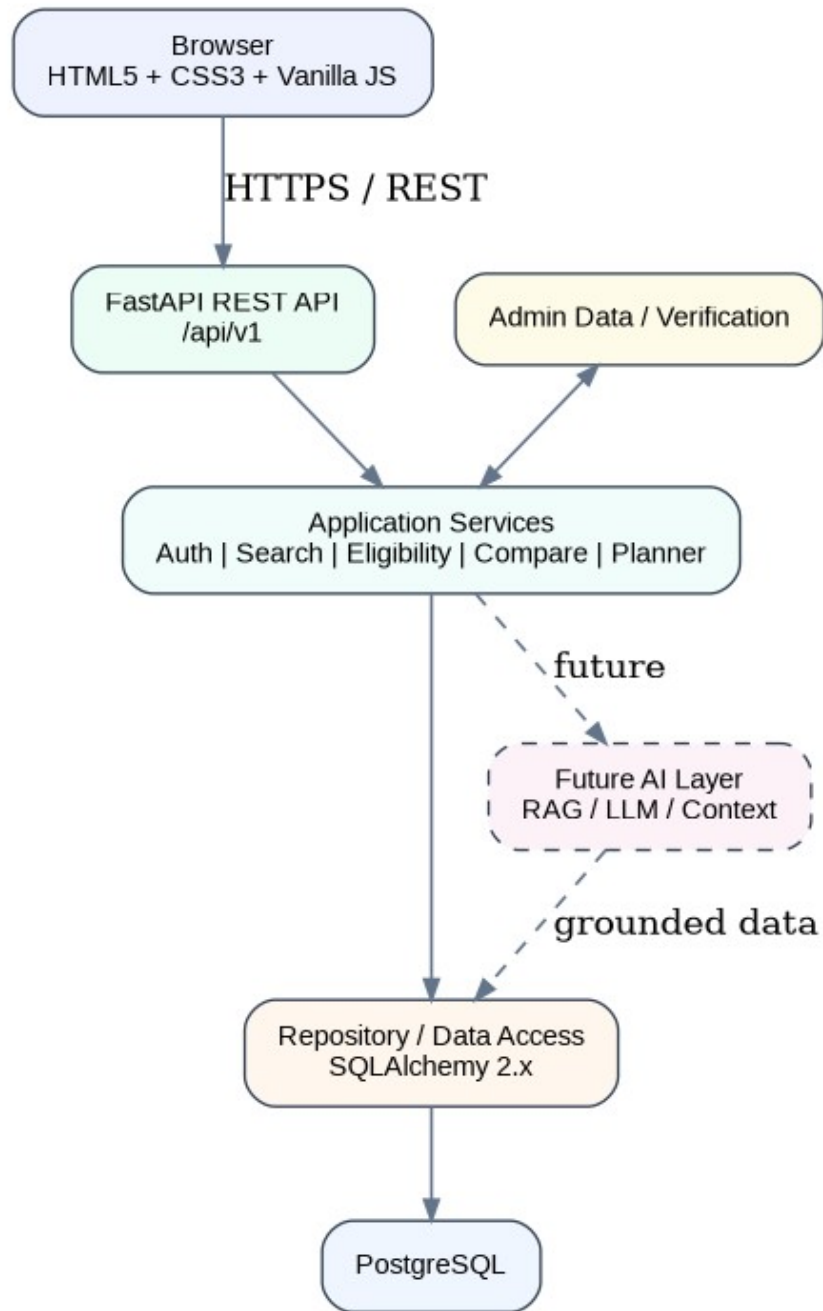


Figure 1. High-level system architecture

1. Executive Summary

The platform is a study-abroad discovery and decision-making workspace. It helps students move from an initial intention to study abroad toward a concrete shortlist and application plan. The product is not intended to be a generic directory or a chatbot-first application.

The primary experience is: Discover -> Explore -> Understand -> Check Fit -> Compare -> Shortlist -> Plan -> Apply.

Core product value

- Structured country, university and course information.
- Clear presentation of eligibility requirements, tuition, deadlines and documents.
- Source-backed academic information with verification metadata.
- Personalized profile and deterministic eligibility evaluation.
- Comparison and shortlist workflows.
- Lightweight application planner.
- Future context-aware AI assistant grounded in the platform data.

Technology

Layer	Technology
Frontend	HTML5, CSS3, Vanilla JavaScript, ES Modules
Backend	Python, FastAPI, Pydantic v2
ORM	SQLAlchemy 2.x
Database	PostgreSQL
Migrations	Alembic
Testing	Pytest, HTTPX
Linting	Ruff
Future AI	RAG/LLM service behind FastAPI

2. Product Principles

- Clarity over information density.
- Source before opinion.
- Progressive disclosure instead of giant forms.
- Personalization without unnecessary complexity.
- No admission or visa guarantees.
- The core product works without AI.
- The database is the source of truth; AI is a later interface over structured data.
- Glass is a material used selectively, not the entire design system.

3. Target Users and Primary User Journey

Primary user

An international student researching countries, universities and courses, often without a strong understanding of admission systems, costs or eligibility rules.

Primary journey

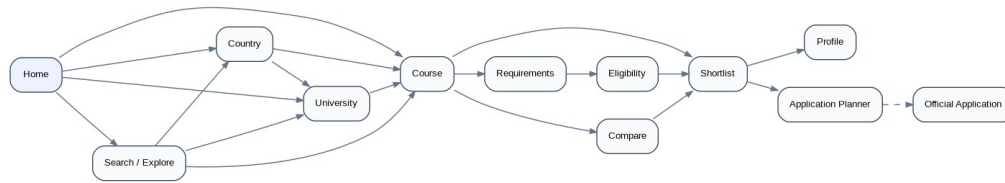


Figure 2. Primary application flow

Canonical decision path

Landing

- > Search / Country
- > University
- > Course
- > Requirements
- > Eligibility
- > Compare
- > Shortlist
- > Profile / Planner
- > Official application

4. Frontend Product Surface

Required pages

Area	Pages
Home	Landing page
Countries	Country listing, country details
Universities	University listing, university details
Courses	Course listing, course details
Decision support	Compare, Shortlist, Eligibility
User	Profile, Application Planner
Auth	Login, Signup, Forgot Password, Verification
System	404, loading, empty, error, outdated states
Admin	Data management and verification interfaces, when implemented

Navigation

Desktop: Logo | Explore | Compare | Shortlist | Search | Theme | Profile

Mobile: Logo | Search | Theme/Menu

Landing page

The landing page is minimal and search-led. It contains a short hero, global search, popular destinations, fields of study, a concise explanation of the workflow, and a trust/source message. Avoid oversized marketing sections.

Course detail page

The course page is the most important page. It must immediately communicate course name, university, location, degree, duration, language, tuition, deadline, requirements, eligibility, source and next actions.

Course header

- Course name
- University
- Location
- Degree / Duration / Language
- Check Eligibility / Save / Compare

Course content

- Overview
- Eligibility
- Requirements
- Tuition and Costs
- Deadlines
- Documents
- Sources

5. UI/UX Design System

Visual direction

- Premium, minimal, calm, editorial and trustworthy.
- Strong typography and whitespace.
- Neutral palette with one restrained accent.
- Soft borders and restrained shadows.
- Selective glass surfaces inspired by premium consumer interfaces.
- No neon gradients, AI robots, giant glowing controls or template-heavy card grids.

Typography

Recommended system stack: -apple-system, BlinkMacSystemFont, SF Pro Display, SF Pro Text, Segoe UI, Roboto, Arial, sans-serif. Exact font licensing must be verified before bundling proprietary font files.

Token	Suggested size	Use
Display	56px	Hero / large editorial heading
H1	48px	Primary page title
H2	36px	Major section
H3	24px	Subsection
Body large	18px	Intro / supporting copy
Body	16px	Standard content
Small	14px	Supporting metadata
Metadata	13px	Compact labels

Spacing

Use a 4px-based scale: 4, 8, 12, 16, 20, 24, 32, 40, 48, 64, 80, 96, 120.

Glass rules

- Use glass for navbar-on-scroll, search overlays, compare tray, sticky course actions, modals, filter sheets and AI panel.
- Do not make every card or section glass.
- Light mode glass: translucent white, subtle dark border, moderate blur.
- Dark mode glass: translucent dark surface, subtle light border, moderate blur.

- Provide fallback styling when backdrop-filter is unavailable.

Light and dark mode

Implement a shared HTML structure using CSS custom properties. Use data-theme="light" and data-theme="dark" on the document root. Default to saved user preference; if absent, use system preference. Persist explicit user choice. Do not invert colors mechanically.

Dark mode readability rule

Dark mode must be designed independently. Never use black text on dark surfaces, overly dim gray body text, or white browser-default form controls. Course requirements, deadlines and long-form content must remain comfortable to read for extended sessions.

6. Responsive Design Requirements

Every element must be responsive: navbar, search, cards, lists, filters, course sidebar, sticky actions, comparison, forms, modals, bottom sheets, AI panel, tables, images, icons and footer.

Responsive breakpoints

Range	Primary behavior
Mobile: ~320-767px	One column, bottom-sheet filters, compact navigation, stacked actions
Tablet: ~768-1023px	Reduced columns, compact filters, flexible sidebars
Desktop: ~1024-1279px	Two-column layouts where useful
Wide: 1280px+	Full content width up to about 1200-1320px

Do not use horizontal overflow as a substitute for responsive design except where a comparison table explicitly benefits from horizontal scrolling. Long course and university names must wrap naturally.

Touch targets

Interactive targets should provide an approximately 44px touch area. This applies to buttons, icon controls, filters, theme toggle, save, compare and menus.

7. Application Flow Details

Discovery

1. User opens Home.
2. User searches or chooses a country.
3. User opens a country, university or course.
4. User reads details and requirements.
5. User optionally creates a profile and checks eligibility.
6. User compares and saves preferred options.
7. User moves saved options into the application planner.

Authentication interruption flow

Course page

-> Save

-> User not authenticated

- > Login / Signup
- > Successful authentication
- > Return to original course
- > Complete Save

Eligibility flow

Course -> Check Eligibility

-> Existing profile?

Yes -> Evaluate

No -> Progressive profile setup -> Evaluate

-> Strong Match / Possible Match / Weak Match / Unknown

8. Backend Architecture

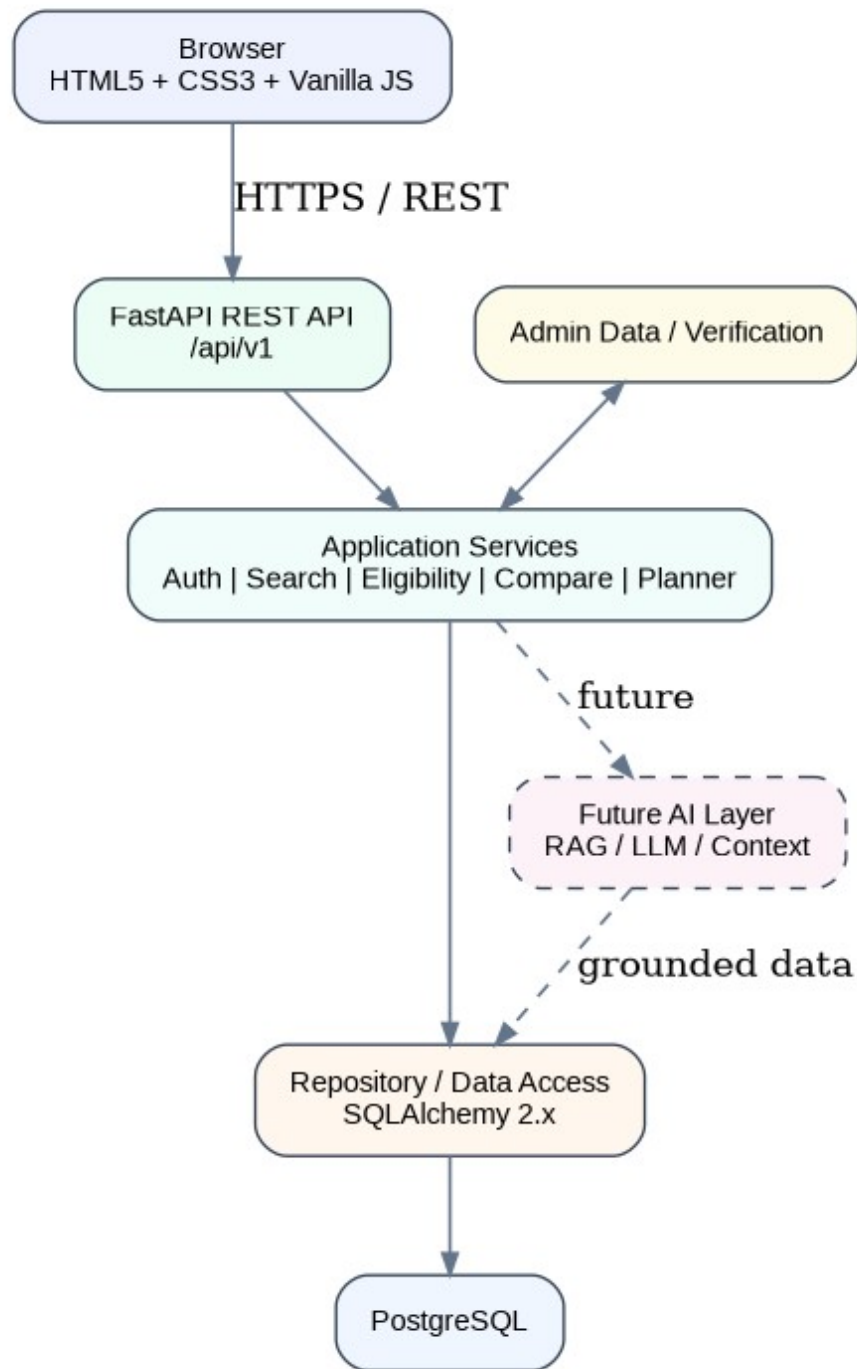


Figure 3. Backend and frontend architecture

Layering

Router

- > Pydantic schema validation
- > Service layer
- > Repository / SQLAlchemy
- > PostgreSQL

- Routers handle HTTP concerns and dependency injection.
- Services contain business rules.
- Repositories isolate database access.
- Pydantic schemas define request/response contracts.
- PostgreSQL remains the authoritative data store.
- The architecture is a modular monolith, not microservices.

9. Backend Project Structure

```
backend/  
  app/  
    main.py  
    config.py  
    database.py  
    dependencies.py  
    security.py  
  api/v1/  
    models/  
    schemas/  
    services/  
    repositories/  
    utils/  
  alembic/  
  tests/  
  alembic.ini  
  requirements.txt  
  .env.example
```

Core dependencies

FastAPI, Uvicorn, Pydantic v2, pydantic-settings, SQLAlchemy 2.x, Alembic, psycopg, Pytest, HTTPX and Ruff.
Avoid unnecessary dependencies.

10. Database and ER Diagram

The database is relational and normalized. UUIDs are recommended for primary keys. Public entities also have human-readable slugs. Authoritative academic records use active status and source verification rather than destructive deletion.

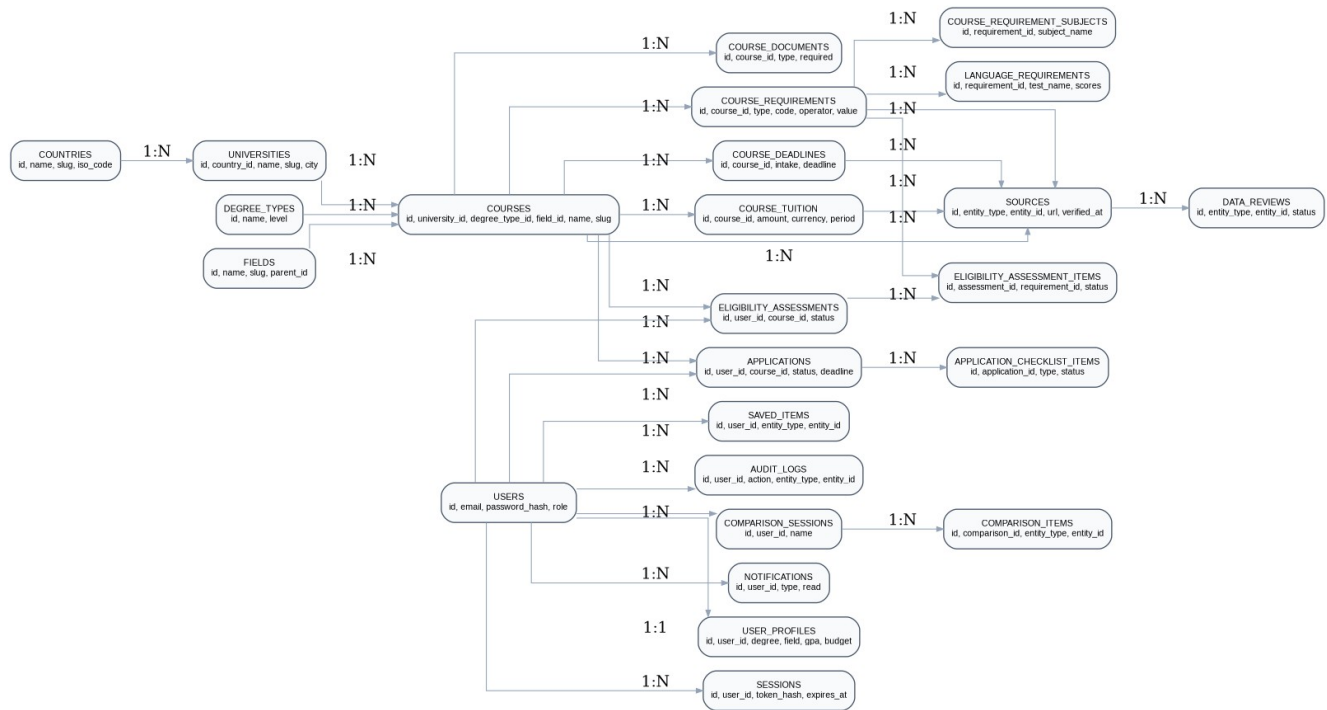


Figure 4. Core entity relationship diagram

Core relationship summary

- Country 1:N University.
- University 1:N Course.
- Degree Type 1:N Course.
- Field 1:N Course, with optional parent hierarchy.
- Course 1:N Requirements, Deadlines, Tuition records and Documents.
- Course requirements can have subject rows and language/test detail rows.
- Course, requirement, deadline and tuition facts can reference source records.
- User 1:1 Profile and 1:N Sessions, Saved Items, Eligibility Assessments, Applications, Notifications and Comparisons.
- Eligibility Assessment 1:N Assessment Items.
- Application 1:N Checklist Items.

11. PostgreSQL Schema - Reference and Academic Data

countries

Column	Type	Notes
id	UUID PK	Primary identifier
name	VARCHAR(120)	Country name
slug	VARCHAR(140) UNIQUE	Public URL key
iso_code	VARCHAR(3) UNIQUE	ISO code
flag_code	VARCHAR(10)	Optional display code
currency_code	VARCHAR(3)	Default currency
description	TEXT	Country overview
study_overview	TEXT	Study guidance
cost_overview	TEXT	Cost guidance
student_info	TEXT	Student information

visa_overview	TEXT	General visa context
is_active	BOOLEAN	Soft publication state
created_at / updated_at	TIMESTAMPTZ	Audit timestamps

country_languages

Column	Type	Notes
id	UUID PK	
country_id	UUID FK	countries.id
language_code	VARCHAR	
language_name	VARCHAR	
is_primary	BOOLEAN	
Unique	country_id + language_code	Avoid duplicates

country_costs

Column	Type	Notes
id	UUID PK	
country_id	UUID FK	countries.id
cost_type	VARCHAR	tuition/living/accommodation/etc.
minimum_amount	NUMERIC	Estimate
maximum_amount	NUMERIC	Estimate
currency_code	VARCHAR	
period	VARCHAR	monthly/yearly/etc.
source_id	UUID FK	Provenance
effective_from / until	DATE	Validity

universities

Column	Type	Notes
id	UUID PK	
country_id	UUID FK	countries.id
name	VARCHAR	Institution name
slug	VARCHAR UNIQUE	Public URL
short_name	VARCHAR	Search alias
logo_url	TEXT	
website_url	TEXT	Official site
city	VARCHAR	
state_region	VARCHAR	
postal_code	VARCHAR	
description	TEXT	
institution_type	VARCHAR	public/private/etc.
founded_year	INTEGER	
latitude / longitude	NUMERIC	Optional
is_active	BOOLEAN	
created_at / updated_at	TIMESTAMPTZ	

degree_types and fields

Entity	Important columns
degree_types	id, name, slug, level, description
fields	id, name, slug, parent_id, description

Fields are hierarchical so Computer Science can have subfields such as AI, Data Science, Software Engineering and Cybersecurity.

12. PostgreSQL Schema - Courses and Requirements

courses

Column	Type	Notes
id	UUID PK	
university_id	UUID FK	universities.id
name	VARCHAR	Course name
slug	VARCHAR UNIQUE	Public URL
degree_type_id	UUID FK	
field_id	UUID FK	
description	TEXT	
duration_value	NUMERIC	
duration_unit	VARCHAR	years/months
study_mode	VARCHAR	full-time/part-time/mixed
attendance_type	VARCHAR	on-campus/online/hybrid
language_notes	TEXT	
credits	NUMERIC	
application_url	TEXT	Official application
official_course_url	TEXT	Official course source
is_active	BOOLEAN	
created_at / updated_at	TIMESTAMPTZ	

course_requirements

Column	Type	Notes
id	UUID PK	
course_id	UUID FK	
requirement_type	VARCHAR	academic/language/test/subject/etc.
requirement_code	VARCHAR	minimum_cgpa, ielts, gre, degree, etc.
requirement_name	VARCHAR	Human label
operator	VARCHAR	eq/gt/gte/lt/lte/contains/one_of/ required/not_required
value_numeric	NUMERIC	Numeric comparison
value_text	TEXT	Text/list comparison
unit	VARCHAR	10, score, years, etc.
is_required	BOOLEAN	
is_strict	BOOLEAN	
notes	TEXT	
source_id	UUID FK	

course_requirement_subjects

Column	Type	Notes
id	UUID PK	
requirement_id	UUID FK	
subject_name	VARCHAR	Prerequisite subject
minimum_credits	NUMERIC	Optional
minimum_grade	VARCHAR	Optional
is_required	BOOLEAN	

notes	TEXT	
-------	------	--

language_requirements

Column	Type	Notes
id	UUID PK	
course_requirement_id	UUID FK	
test_name	VARCHAR	IELTS/TOEFL/PTE
overall_min_score	NUMERIC	
reading_min_score	NUMERIC	
writing_min_score	NUMERIC	
listening_min_score	NUMERIC	
speaking_min_score	NUMERIC	
validity_months	INTEGER	Optional
source_id	UUID FK	

course_deadlines

Column	Type	Notes
id	UUID PK	
course_id	UUID FK	
intake_name	VARCHAR	
intake_year	INTEGER	
deadline_type	VARCHAR	application/scholarship/etc.
deadline_date	DATE	
timezone	VARCHAR	
is_final	BOOLEAN	
is_open	BOOLEAN	
source_id	UUID FK	

course_tuition

Column	Type	Notes
id	UUID PK	
course_id	UUID FK	
amount	NUMERIC	
currency_code	VARCHAR	
period	VARCHAR	semester/year/etc.
tuition_type	VARCHAR	
student_category	VARCHAR	domestic/EU/international/all
notes	TEXT	
source_id	UUID FK	
effective_from / until	DATE	
is_current	BOOLEAN	

course_documents

Column	Type	Notes
id	UUID PK	
course_id	UUID FK	
document_type	VARCHAR	passport/transcript/CV/etc.
document_name	VARCHAR	

description	TEXT	
is_required	BOOLEAN	
submission_format	VARCHAR	
source_id	UUID FK	

13. PostgreSQL Schema - Provenance, Data Quality and Trust

sources

Column	Type	Notes
id	UUID PK	
source_type	VARCHAR	official university/government/testing body/trusted external
source_name	VARCHAR	
source_url	TEXT	
entity_type	VARCHAR	course/requirement/deadline/etc.
entity_id	UUID	Referenced record id
published_at	TIMESTAMPTZ	Optional
last_verified_at	TIMESTAMPTZ	Verification timestamp
verification_status	VARCHAR	verified/needs_review/outdated/unavailable
verified_by	UUID	Admin/editor user
verification_notes	TEXT	

data_reviews

Column	Type	Notes
id	UUID PK	
entity_type / entity_id	VARCHAR/UUID	What is being reviewed
review_type	VARCHAR	
reason	TEXT	
assigned_to	UUID	Reviewer
status	VARCHAR	pending/in_review/verified/rejected
reviewed_at	TIMESTAMPTZ	
notes	TEXT	

audit_logs

Column	Type	Notes
id	UUID PK	
user_id	UUID FK	Actor
action	VARCHAR	create/update/delete/verify/publish/etc.
entity_type	VARCHAR	
entity_id	UUID	
old_values	JSONB	Prior state
new_values	JSONB	New state
ip_address	INET/VARCHAR	
user_agent	TEXT	
created_at	TIMESTAMPTZ	

Trust model

Every important factual value should be traceable to a source, with a verification status and timestamp. The UI must

distinguish verified facts, estimates, unavailable information and AI interpretation.

14. PostgreSQL Schema - User, Authentication and Sessions

users

Column	Type	Notes
id	UUID PK	User identifier
email	VARCHAR UNIQUE	Normalized lowercase email
password_hash	TEXT	Argon2id/bcrypt hash only
auth_provider	VARCHAR	local/google/etc.
provider_user_id	VARCHAR nullable	OAuth identity
role	VARCHAR	user/editor/admin
is_verified	BOOLEAN	Email verification state
is_active	BOOLEAN	Account state
created_at / updated_at	TIMESTAMPTZ	
last_login_at	TIMESTAMPTZ	

sessions

Column	Type	Notes
id	UUID PK	
user_id	UUID FK	users.id
session_token_hash	TEXT	Never store raw token
expires_at	TIMESTAMPTZ	
created_at	TIMESTAMPTZ	
last_seen_at	TIMESTAMPTZ	
revoked_at	TIMESTAMPTZ nullable	
ip_address	INET/VARCHAR	
user_agent	TEXT	

user_profiles

Column	Type	Notes
id	UUID PK	
user_id	UUID UNIQUE FK	One profile per user
first_name	VARCHAR	
last_name	VARCHAR	
country_of_residence	VARCHAR	
current_degree	VARCHAR	
field	VARCHAR	
current_university	VARCHAR	
graduation_year	INTEGER	
gpa	NUMERIC	
gpa_scale	NUMERIC	
work_experience_years	NUMERIC	
research_experience_years	NUMERIC	
target_degree	VARCHAR	
preferred_intake	VARCHAR	
budget_amount	NUMERIC	
budget_currency	VARCHAR	

created_at / updated_at	TIMESTAMPTZ	
-------------------------	-------------	--

Authentication flow

Signup

- > validate request
- > normalize email
- > hash password
- > insert user
- > insert profile
- > create session
- > set HTTP-only cookie

Login

- > normalize email
- > find user
- > verify password hash
- > create session
- > set HTTP-only cookie
- > update last_login_at

Logout

- > identify session
- > revoke session
- > clear cookie

15. PostgreSQL Schema - Personal Decision Support

saved_items

Column	Type	Notes
id	UUID PK	
user_id	UUID FK	
entity_type	VARCHAR	country/university/course/scholarship
entity_id	UUID	Referenced entity
created_at	TIMESTAMPTZ	
Unique	user_id + entity_type + entity_id	Prevent duplicates

comparison_sessions and comparison_items

Table	Important columns
comparison_sessions	id, user_id, name, created_at, updated_at
comparison_items	id, comparison_id, entity_type, entity_id, position, created_at

eligibility_assessments

Column	Type	Notes
id	UUID PK	
user_id	UUID FK	
course_id	UUID FK	
overall_status	VARCHAR	strong_match/possible_match/ weak_match/unknown
matched_count	INTEGER	

unmatched_count	INTEGER	
unknown_count	INTEGER	
profile_snapshot	JSONB	Inputs used to reproduce result
assessment_version	VARCHAR	Rule version
created_at	TIMESTAMPTZ	

eligibility_assessment_items

Column	Type	Notes
id	UUID PK	
assessment_id	UUID FK	
requirement_id	UUID FK	
status	VARCHAR	matched/unmatched/unknown/ not_applicable
user_value	TEXT/JSONB	
required_value	TEXT/JSONB	
explanation	TEXT	Human-readable reason
created_at	TIMESTAMPTZ	

applications

Column	Type	Notes
id	UUID PK	
user_id	UUID FK	
course_id	UUID FK	
intake_id	UUID nullable	
status	VARCHAR	researching/shortlisted/preparing/ ready/applied/etc.
application_deadline	DATE	
notes	TEXT	
submitted_at	TIMESTAMPTZ	
decision_date	TIMESTAMPTZ	
created_at / updated_at	TIMESTAMPTZ	

application_checklist_items

Column	Type	Notes
id	UUID PK	
application_id	UUID FK	
document_type	VARCHAR	
label	VARCHAR	
description	TEXT	
is_required	BOOLEAN	
status	VARCHAR	pending/in_progress/completed/ not_required
completed_at	TIMESTAMPTZ	
notes	TEXT	

notifications

Column	Type	Notes
id	UUID PK	

user_id	UUID FK	
notification_type	VARCHAR	deadline/data_update/application/system
title	VARCHAR	
message	TEXT	
reference_type	VARCHAR	
reference_id	UUID	
is_read	BOOLEAN	
read_at	TIMESTAMPTZ	
created_at	TIMESTAMPTZ	

16. API Design

API base

/api/v1

Authentication

Method	Endpoint	Purpose
POST	/api/v1/auth/register	Create user + profile + session
POST	/api/v1/auth/login	Authenticate and create session
POST	/api/v1/auth/logout	Revoke session
GET	/api/v1/auth/me	Return current authenticated user
POST	/api/v1/auth/forgot-password	Future password reset start
POST	/api/v1/auth/reset-password	Future reset completion
POST	/api/v1/auth/verify-email	Future email verification

Profile

Method	Endpoint	Purpose
GET	/api/v1/profile	Read current profile
PUT	/api/v1/profile	Update current profile

Future academic APIs

Domain	Example endpoints
Countries	GET /api/v1/countries, GET /api/v1/countries/{slug}
Universities	GET /api/v1/universities, GET /api/v1/universities/{slug}
Courses	GET /api/v1/courses, GET /api/v1/courses/{slug}
Search	GET /api/v1/search?q=...
Requirements	GET /api/v1/courses/{id}/requirements
Deadlines	GET /api/v1/courses/{id}/deadlines
Sources	GET /api/v1/courses/{id}/sources

API response convention

```
{
  "data": {...},
  "meta": {...}
}
```

```
{
  "error": {
```

```

"code": "ERROR_CODE",
"message": "Human readable message"
}
}

```

17. Security Architecture

- Passwords are hashed using Argon2id or another modern memory-hard algorithm. Plain-text passwords never reach storage or logs.
- Authentication uses a secure HTTP-only session cookie. Raw session tokens are never stored in the database.
- Production cookies use Secure and an appropriate SameSite policy.
- CORS is explicit; wildcard origins are not used with credentials.
- Authorization is checked on the server. The frontend cannot decide user identity or privileges.
- Sensitive fields and authentication secrets are excluded from logs.
- Login and password reset endpoints receive rate limiting.
- Input validation and database constraints protect against malformed data and duplicate accounts.
- State-changing cookie-authenticated requests should use appropriate CSRF protection when required by the deployment model.

Critical rule

Never trust the browser for identity

A frontend request must not be allowed to select another user by sending a `user_id`. FastAPI derives the authenticated identity from the session cookie and applies server-side authorization.

18. Performance and Reliability

- Avoid infinite loops, recursive renders, request storms and duplicate initialization.
- Use page-specific initialization rather than running all page modules on every page.
- Debounce search input and abort obsolete requests where useful.
- Do not use heavy backdrop blur over large areas.
- Prefer CSS sticky behavior over continuous scroll JavaScript when possible.
- Use pagination and server-side filtering instead of downloading thousands of records.
- Use loading, empty and error states for every data-driven screen.
- Do not hide errors; diagnose the root cause.
- Keep the main thread responsive. Browser freeze is a release-blocking defect.

Performance debugging order

1. Infinite loops
2. Duplicate initialization
3. Repeated API calls
4. Excessive DOM rendering
5. Expensive synchronous JS
6. Large images
7. Expensive CSS effects
8. Decorative animation

19. Authentication and Frontend Integration

Cookie-based browser flow

Frontend signup

- > POST /auth/register
- > FastAPI
- > PostgreSQL
- > Set-Cookie: study_abroad_session

Frontend refresh

- > GET /auth/me with credentials: include
- > FastAPI validates session
- > User restored

Frontend auth state

- Keep current user and authentication state in a small ES module.
- Do not store raw session tokens in localStorage or sessionStorage.
- Use credentials: include in API requests that rely on cookies.
- Restore user state from /auth/me after refresh.
- Clear local auth state after successful logout or session expiration.
- Prevent double-submit of login, signup and profile forms.

20. Error, Empty and Loading States

State	Behavior
Loading	Skeleton or lightweight loading indicator
Empty	Explain why there are no records and offer next action
Error	Explain failure and provide retry when appropriate
Unavailable	State that information is currently unavailable
Outdated	Warn that information may have changed and point to official source
Success	Provide concise confirmation after server-side success

21. Admin and Data Verification

Admin capabilities

- Create/edit countries, universities and courses.
- Edit requirements, tuition and deadlines.
- Attach official sources.
- Mark content verified, needs review, outdated or unavailable.
- Review data quality issues.
- Publish/unpublish records.
- Inspect audit logs.

Publishing flow

Draft -> Review -> Verified -> Published

Published -> Needs Review -> Re-verified -> Published

Published -> Outdated when source validity expires

22. Implementation Plan

Phase 0 - Repository foundation

- Monorepo structure and documentation.
- Backend environment and requirements.
- Frontend HTML/CSS/JS shell.
- GitHub Actions CI.
- README and .gitignore.

Phase 1 - Backend foundation

- FastAPI application.
- Configuration.
- PostgreSQL connection.
- SQLAlchemy base and session dependency.
- Alembic initialization.
- Health endpoint.
- Request ID, CORS and error handling.

Phase 2 - Database

- Create reference tables.
- Create country/university/course schema.
- Create requirements/deadlines/tuition/documents.
- Create sources and data review tables.
- Create user/auth/profile tables.
- Create saved/compare/eligibility/application tables.
- Seed a small verified dataset.

Phase 3 onward

- Public APIs and search.
- Frontend design system and connected discovery experience.
- Authentication integration.
- Eligibility engine.
- Compare and shortlist.
- Application planner and notifications.
- Admin verification.
- Production hardening.
- AI layer last.

23. AI Integration Plan - Future

AI is a later layer, not the source of truth. The platform should first create reliable structured data and deterministic decision logic.

Course page

-> contextual AI request

-> FastAPI

-> retrieve course + requirement + source + user profile

-> AI service
 -> answer + sources
 -> frontend assistant panel

- AI should distinguish official requirements, estimates and interpretation.
- AI should not override deterministic eligibility rules.
- AI answers should expose relevant sources.
- AI usage should be rate-limited and tracked.
- The core product must remain useful if the AI service is unavailable.

24. Recommended Frontend Structure

```
frontend/
  index.html
  pages/
  css/
    tokens.css
    reset.css
    typography.css
    layout.css
    components.css
    forms.css
    responsive.css
  js/
    main.js
    api/
    components/
    pages/
    state/
    utils/
  assets/
```

Reusable components

- Navbar
- Search
- Country item
- University item
- Course item
- Requirement row
- Source info
- Eligibility summary
- Compare tray
- Modal
- Bottom sheet
- Toast
- Skeleton
- Breadcrumb

25. Development and QA Checklist

Functional

- All major links work.
- Detail pages receive stable identifiers.
- Search and filters work.
- Save and compare interactions work.
- Profile persistence works.
- Signup and login persist user data in PostgreSQL.
- Refresh restores authenticated state.
- Logout revokes the session.

Responsive

- Mobile, tablet and desktop layouts are intentional.
- No accidental horizontal overflow.
- Filter sidebars convert to bottom sheets on mobile.
- Comparison remains readable on small screens.
- Sticky actions do not cover content.

Theme

- Light mode readable and coherent.
- Dark mode readable and coherent.
- Theme preference persists.
- No flash of incorrect theme.
- Inputs, tables, icons, glass, modals and status colors adapt.

Reliability

- No uncaught JavaScript errors.
- No infinite request loops.
- No duplicate initialization.
- No page freeze.
- API failures produce bounded, retryable errors.
- Database migrations are reproducible.

26. Definition of Done

The foundation is considered complete when the frontend, FastAPI backend and PostgreSQL database work as one system, authentication is real and persistent, and the core user journey can be executed without broken navigation or browser freezes.

Browser

<-> Vanilla JS frontend
 <-> FastAPI REST API
 <-> SQLAlchemy
 <-> PostgreSQL

Authentication

signup -> users + profile + session

login -> verify hash + session
me -> restore user
logout -> revoke session

27. Final Architecture Principles

- Build the decision engine before the intelligence layer.
- Use PostgreSQL as the authoritative source of user and academic data.
- Use source and verification metadata for trust.
- Keep the frontend framework-free and modular.
- Keep authentication secure and cookie-based.
- Keep business rules on the backend.
- Use deterministic eligibility for hard requirement checks.
- Use AI later as a grounded explanation and discovery layer.
- Prefer a modular monolith over premature microservices.
- Fix functional reliability before visual polish.

One-sentence product architecture

Reliable study-abroad data + clear UX + deterministic decision support + secure user persistence, with AI added later as a contextual interface over the trusted data layer.